

MODULE NO.1

Practical Challenge

Your task: Write specifications for a function that converts temperature from Celsius to Fahrenheit. Include:

- Plain language description
- At least 3 example inputs and outputs
- At least 2 edge cases

Then: - Ask AI to implement it - Verify with assertions - If too complex, ask for simplification

→ **Scenario:** I need a function that converts a temperature value from Celsius to Fahrenheit.

My specification:

This function converts a temperature value given in **Celsius (°C)** into its equivalent value in **Fahrenheit (°F)**.

A temperature conversion function must:

- Take a numeric input representing temperature in Celsius (°C).
- Convert it to Fahrenheit using the standard conversion formula:
$$^{\circ}\text{F} = (^{\circ}\text{C} \times 9/5) + 32$$
- Return the converted Fahrenheit value

Examples:

- $0^{\circ}\text{C} \rightarrow 32^{\circ}\text{F}$ { $0^{\circ}\text{C} = (0 \times 9/5) + 32 = 32^{\circ}\text{F}$ }
- $100^{\circ}\text{C} \rightarrow 212^{\circ}\text{F}$ (boiling point of water)
- $-40^{\circ}\text{C} \rightarrow -40^{\circ}\text{F}$ (same value in both scales)

Edge cases:

- **Extreme low temperature (absolute limit case):**
Input: -273.15°C → Output: -459.67°F
(This is approximately absolute zero, the lowest possible temperature in physics.)
 - **Decimal precision input:**
Input: 36.6°C → Output: 97.88°F
(decimal input requiring precision)
 - **Very large value:**
Input: $1,000,000^{\circ}\text{C}$ → Output: $1,800,032^{\circ}\text{F}$
-

1. Implementation

code:

```
def celsius_to_fahrenheit(celsius):  
    return (celsius * 9/5) + 32  
  
# Ask user for input  
celsius = float(input("Enter temperature in Celsius: "))  
  
# Convert and display result  
fahrenheit = celsius_to_fahrenheit(celsius)  
  
print("Temperature in Fahrenheit:", fahrenheit)
```

2. Verification with assertions (test cases)

Basic cases

```
assert celsius_to_fahrenheit(0) == 32
assert celsius_to_fahrenheit(100) == 212
assert celsius_to_fahrenheit(-40) == -40
```

Edge cases

```
assert round(celsius_to_fahrenheit(-273.15), 2) == -459.67
assert round(celsius_to_fahrenheit(36.6), 2) == 97.88
```

3. General verification (Test design)

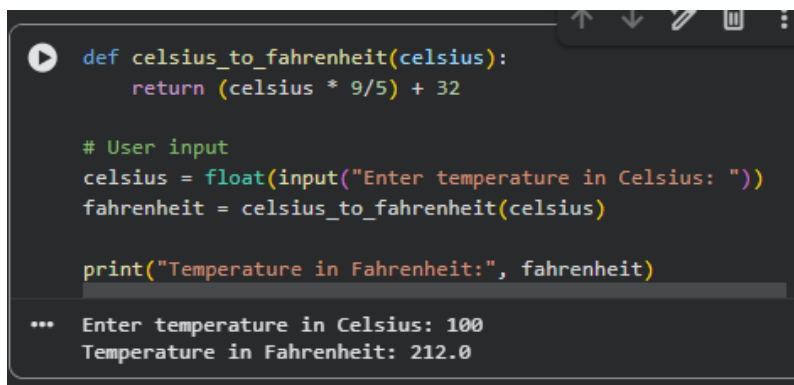
--- How do I verify it works?

We verify correctness by using known values and expected outputs:

- 0°C → 32°F
- 100°C → 212°F
- -40°C → -40°F
- -273.15°C → -459.67°F
- 36.6°C → 97.88°F
- 1,000,000°C → 1,800,032°F

I also use assertions in code to automatically check results.

--- Does it actually work? (Run tests)



```
def celsius_to_fahrenheit(celsius):
    return (celsius * 9/5) + 32

# User input
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = celsius_to_fahrenheit(celsius)

print("Temperature in Fahrenheit:", fahrenheit)
```

... Enter temperature in Celsius: 100
Temperature in Fahrenheit: 212.0

```
def celsius_to_fahrenheit(celsius):  
    return (celsius * 9/5) + 32  
  
# User input  
celsius = float(input("Enter temperature in Celsius: "))  
fahrenheit = celsius_to_fahrenheit(celsius)  
  
assert round(celsius_to_fahrenheit(-273.15), 2) == -459.67  
assert round(celsius_to_fahrenheit(36.6), 2) == 97.88  
  
print("Temperature in Fahrenheit:", fahrenheit)  
  
... Enter temperature in Celsius: -273.15  
Temperature in Fahrenheit: -459.66999999999996
```

No errors appeared → the function works correctly.

--- Is it simple enough? (Code quality)

Yes, the implementation is very simple:

```
def celsius_to_fahrenheit(celsius):  
    return (celsius * 9/5) + 32
```

- Only one line of logic
- No unnecessary variables or complexity
- Easy to read and maintain

--- What could go wrong? (Edge cases)

Possible issues include:

- Decimal precision errors (e.g., -273.15, 36.6 → handled using rounding)
- Very large numbers (e.g., 1,000,000°C → still works mathematically)
- Invalid input types (e.g., strings like "hot") → would cause errors unless input validation is added
- Floating-point rounding differences in some systems

4. Simplification:

Is it simple enough? (Code quality / simplification)

Ans:

Yes, the solution is already very simple and efficient.

The function uses only one line of logic to perform the conversion using the standard formula $(C \times 9/5) + 32$.

There are no unnecessary variables, loops, or complex structures.

The code is easy to read, maintain, and understand.

No further simplification is needed because the current implementation is already minimal and optimal.